Risk Assessment Report

Table of Contents

1. Executive Summary

Deployment of the Installation Robot for Contested Environments Software (CIRCESoft) presents unique security benefits and challenges as it is intended for operation in high-stakes, mission-critical environments. Thus, CIRCESoft requires high availability (HA) and robust hardening to minimize the risk of data disclosure or denial of service. CIRCESoft's counterpart, CIRCEBot, operates as a cyber-physical system (CPS) and thus is subjected to many different real-world scenarios that both systems must work together on to ensure continued successful operation of CIRCEBot. This document details the attack surface, including security risks and mitigations, CIRCESoft while delivering a lighter overview of the attack surface of CIRCEBot.

1.1. Purpose

The Cable Installation Robots for Contested Environments Software (CIRCESoft) is the Human-Machine Interface (HMI) for CIRCEBot. CIRCEBot's purpose is to lay up to 300ft of Ethernet cable to establish a reliable command and control (C2) connection between rear and forward installations in a contested environment. CIRCESoft is responsible for calculating the shortest possible path for CIRCEBot to a location, avoiding obstacles and staying within the maximum operational range (300ft) during traversal.

While CIRCEBot can function independently of CIRCESoft once a path is given to it thanks to onboard LiDAR sensors and a stored path functionality, CIRCESoft provides the operator with

valuable telemetry data to ensure the continued safe performance of CIRCEBot. A message panel shows information and warnings from CIRCEBot, while a progress bar and information panel show remaining cable, battery life, and distance to the intended destination.

1.2. Summary of Key Findings

Barring the connection to CIRCEBot, the lack of connections to external services (cloud compute, online resources, etc.) allow CIRCESoft to operate in an airgapped setting which greatly increases the reliability and minimizes the attack surface of the software. Communications between the frontend, backend, and A* pathfinding algorithm all occur over localhost, denying exposure of critical information to the Internet and threat actors. For a threat actor to compromise the integrity of CIRCESoft, they would require a physical connection to the device in order to gain a foothold within the software. Given that CIRCESoft would be operated within military installations, we believe the risk of this occurring is minimal.

The largest point of attack, however, is the Ethernet cable itself. A threat actor could hypothetically splice a malicious connection onto the wire and act as a man in the middle, intercepting valuable C2 information. This necessitates a high degree of security and low degree of trust in the connection and its security.

1.3. Recommendations

We recommend the following actions be taken to ensure the security of the Ethernet cable:

1. Use a camoflauge pattern on the cable to obscure it from ordinary view of attackers.
2. Monitor for losses of connectivity and stop sensitive communication after a loss of connectivity occurs until inspection of the cable can occur, since downtime could mean that the cable is being tampered with.
3. Require secure keys to be transmitted by the receiving base of the cable before initiating communication in order to ensure that a threat actor is not acting as the receiver.

In future iterations of CIRCESoft, we recommend the following development specifications:

1. Implement end-to-end encryption via secure protocol between CIRCESoft and CIRCEBot since intercepted path data could expose the positions of the origin, destination, and C2 cable.
2. If a wireless connection is used to connect to CIRCEBot, ensure a method of validating CIRCESoft to CIRCEBot and vice versa is established to provide non-repudiation and prevent threat actors from mimicking either entity.

2. Hardware Safety Restrictions

2.1. Insecure Operator Device

The operator's device performs all backend computation and acts as an authoritative controller for CIRCEBot. Because of this, it represents a high-value target. The device must be hardened, isolated, and protected with strict network access controls. Any compromise of the operator device could result in unauthorized command injection, path manipulation, or complete loss of control over CIRCEBot.

2.2. On-Board Wireless Adapters

Although CIRCEBot communicates exclusively over tethered Ethernet, the Raspberry Pi's built-in Wi-Fi and Bluetooth radios remain potential attack vectors if not disabled. An adversary could attempt to establish unauthorized wireless connections or exploit vulnerabilities in the wireless stack, allowing them to interfere with CIRCEBot's internal processes or inject malicious traffic into the system.

2.3. Easily Accessible Internals

CIRCEBot's modular, hot-swappable design improves usability but significantly increases physical attack risk. Direct access to the on-board computer allows an adversary to modify the Raspberry Pi's filesystem, replace the SD card, install malicious peripherals, or introduce unauthorized code. Any such compromise could result in the bot sending falsified telemetry, executing adversarial commands, or attempting to infect the operator's device through the tethered Ethernet connection.

3. Future Recommendations

3.1. External Hosting/Network Security

The current API architecture is meant for an air-gapped network for quick deployment and only between trusted parties.

3.2. Cable Hijacking

This refers to vulnerabilities where the physical constraint of the robot's cable length is bypassed or manipulated, potentially leading to equipment damage or a faulty path.

A. Path Limit Enforcement Bypass

- **Pathfinder Check:** The `grid_astar.py` worker correctly enforces the hard constraint that the path length must not exceed **300.0 ft** (`MAX_CABLE_FT`) before writing the path to `grid_path.json`.
- **Vulnerability:** If a malicious client or a system failure allows a path to be *manually* written to the `/grid/path` endpoint without going through the A* worker, the 300 ft limit could be ignored.
- **Mitigation:** The FastAPI server's **`PUT /grid/path` endpoint should implement server-side validation** to re-calculate the total path length and reject any path exceeding `MAX_CABLE_FT` before writing it to the file. This creates a redundant safety check.

B. Status Falsification (Real-time Cable Tracking)

- **Current Tracking:** The `utils.py` module tracks **`CABLE_REMAINING`** by subtracting the Euclidean distance traveled between ECI coordinates reported to the `/current-values` endpoint.
- **Vulnerability:** An attacker or a malfunctioning robot could continuously send false `current_values` (via the `PUT /current-values` endpoint) that report a position change without a corresponding decrease in cable remaining, or, conversely, a massive jump in position that the system might ignore or incorrectly handle.
- **Mitigation:**
    - **Source Verification:** Implement authentication on the `PUT /current-values` endpoint to ensure only the authenticated robot/telemetry source can update its status.

- **Plausibility Checks:** Add logic in `utils.py` to validate the incoming status: if the distance between the `LAST_POSITION_ECI` and the new reported position is physically impossible given the time delta and the robot's known speed limits, the update should be **rejected**, and an error flag should be raised.

3.3. Input Validation

Since CIRCEBot cannot be actively monitored by a human while traversing through a contest environment due to safety reasons, it must be assumed that CIRCEBot could be compromised and could begin sending malicious data. It is also critical to ensure identity validation, i.e. that another entity cannot mimic CIRCEBot in order to send malicious requests. This assessment recommends that data should never be sent to CIRCEBot unless CIRCEBot's identity can first be confirmed to prevent disclosure of the destination, origin, or communication cable locations. It is also recommended to ensure that API endpoints are rate-limited to prevent denial of service, and that inputs are properly sanitized to avoid remote code execution (RCE) and other vulnerabilities.

JP Ognibene

Gabriwl Adams

Ryan Naleway

Celia Hough

Elias Brady

Joze Lindquist